**وزارت علوم، تحقیقات و فناوری**



**دانشکده علوم ریاضی**

# تحلیل و ارزیابی روش‌های نمونه‌گیری در رگرسیون لجستیک چندجمله‌ای بیزی برای تشخیص تومور مغزی


**نام استاد: جناب دکتر مجتبی گنجعلی**

**نام دانشجو: خانم ناهید حاتمی بلاسی**

**شماره دانشجویی: ۴۰۴۴۲۲۲۰۷**


**تابستان ۱۴۰۵**



**این اثر با حمایت جناب دکتر مجتبی گنجعلی انجام گرفته است.**

چکیده :

در این پژوهش، کارایی چهار روش نمونه‌گیری بیزی شامل الگوریتم متروپولیس– هاستینگز تطبیقی، افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی، افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش بیضوی و رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما در برازش مدل رگرسیون لجستیک چندکلاسه بر روی دادگان تصاویر MRI مغز (شامل چهار کلاس: گلیوما، مننژیوما، بدون تومور و هیپوفیز) مورد مقایسه قرار گرفت. بدین منظور، ابتدا بردار ویژگی هر تصویر با ترکیب ویژگی‌های بافت محلی، گرادیان‌های جهت‌دار و آماره‌های شدت روشنایی استخراج شد. سپس با استفاده از تحلیل مؤلفه‌های اصلی، ابعاد ویژگی‌ها به ۸۰ مؤلفه برتر کاهش یافت. هر چهار الگوریتم نمونه‌گیری با تنظیمات یکسان در زنجیره مارکوف (شامل ۶۰۰۰ تکرار دوره سوختگی، ۶۰۰۰ نمونه نگه‌داشته‌شده و نرخ نازک‌سازی با گام ۵ بر روی ۴۱۹۷ داده آموزشی اجرا شدند. ارزیابی مدل‌ها نیز روی ۱۳۹۹ داده اعتبارسنجی و ۱۶۰۰ داده آزمون صورت گرفت. یافته‌های تجربی نشان می‌دهد که روش پلیا–گاما از نظر شاخص اندازه نمونه مؤثر بر ثانیه به‌طور محسوسی کاراترین عملکرد را دارد. با این حال، از منظر معیار دقت روی داده‌های آزمون، روش افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی با ثبت دقت ۷۹.۱۲٪ بهترین عملکرد پیش‌بینی را به ثبت رساند. در ادامه این گزارش، جداول عددی و نمودارهای مرتبط با کارایی روش‌هاینمونه‌گیری زنجیره مارکوف–مونت‌کارلو، بررسی همگرایی زنجیره‌ها و معیارهای دقت طبقه‌بندی ارائه و تحلیل شده و در پایان، جمع‌بندی و پیشنهادهایی برای توسعه پژوهش مطرح خواهد شد.

**واژگان کلیدی** : رگرسیون لجستیک چندکلاسه بیزی، تصاویر تومور مغزی، زنجیره مارکوف مونت‌کارلو، متروپولیس– هاستینگز تطبیقی، روش نمونه‌گیری برش بیضوی، نمونه‌گیری پلیا–گاما، افزایش داده

# فهرست مطالب


۱- مقدمه ..... ۷

۲- مدل‌های پژوهش ..... ۹

۲-۱ رگرسیون چندجمله‌ای بیزی ..... ۹

۲-۲ مدل لینک لاجیت چندجمله‌ای ..... ۹

۲-۳ افزایش داده با بهره‌گیری از متغیرهای توزیع گاما ..... ۱۰

۲-۴الگوریتم متروپولیس– هاستینگز تطبیقی ..... ۱۱

۲-۵افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی ..... ۱۱

۲-۶افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش بیضوی ..... ۱۲

۲-۷رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما ..... ۱۲

۳- معیارهای ارزیابی عملکرد مدل ..... ۱٤

۳-۱ اندازه نمونه مؤثر ..... ۱٤

۳-۲ نرخ نمونه مؤثر ..... ۱٥

٤ -تحلیل داده‌های تصاویر تومور مغزی ..... ۱٥

جدول۱- توزیع تعداد نمونه‌ها بر اساس کلاس و مجموعه داده ..... ۱٦

۵- نتیجه‌گیری ..... ۲۳

## فهرست علائم و اختصارها

$MCMC$ : روش‌های‌نمونه‌گیری زنجیره مارکوف–مونت‌کارلو

$ESS$ : اندازه نمونه مؤثر

$ESR$ : نرخ نمونه مؤثر

$HOG$ : هیستوگرام گرادیان‌های جهت‌دار

$LBP$: الگوهای باینری محلی

## ۱- مقدمه

رگرسیون لجستیک یکی از پرکاربردترین روش‌ها در مسائل طبقه‌بندی احتمالاتی به‌شمار می‌رود؛ مسئله‌ای که در آن هدف، تعیین احتمال تعلق یک مشاهده به یکی از طبقات ممکن است. در مقایسه با بسیاری از روش‌های یادگیری ماشین که غالباً به‌صورت جعبه‌سیاه عمل می‌کنند، رگرسیون لجستیک از مزیت مهم تفسیرپذیری برخوردار است. در این چارچوب، رابطه میان متغیرهای ورودی و احتمال وقوع رویداد مورد نظر به‌صورت صریح قابل بیان و تحلیل است، زیرا هسته اصلی مدل بر پایه ساختاری جمع‌پذیر و به‌طور مشخص یک مدل خطی برای لگاریتم نسبت شانس‌ها بنا شده است. در بسیاری از مسائل کاربردی، متغیر پاسخ تنها به دو طبقه محدود نمی‌شود و ممکن است شامل چندین طبقه باشد. در چنین شرایطی، تعمیم طبیعی رگرسیون لجستیک، رگرسیون لجستیک چندجمله‌ای است.

در رویکردهای بیزی برای برآورد ضرایب این مدل، امکان کمی‌سازی عدم‌قطعیت پارامترها فراهم می‌شود. با این حال، به دلیل وجود جمله غیرخطی نرمال‌ساز در مخرج تابع سافت‌مکس ، توزیع پسین ضرایب معمولاً فرم تحلیلی بسته و مشخصی ندارد. در نتیجه، برای نمونه‌گیری از توزیع پسین ناچار به استفاده از روش‌های زنجیره مارکوف مونت‌کارلو  هستیم که پیاده‌سازی استاندارد آن‌ها مانند الگوریتم متروپولیس–هستینگز نیازمند تنظیم دستی پارامترهای پیشنهاد بوده و ممکن است از نظر محاسباتی پرهزینه یا با مشکل نرخ پذیرش پایین مواجه شوند.

برای مواجهه با این چالش محاسباتی، استفاده از روش‌های افزایش داده  به عنوان راهکاری کارآمد مطرح شده است. در این رویکرد، با وارد کردن متغیرهای کمکی پنهان به مدل، توزیع‌های شرطی پسین ضرایب ساختار ساده‌تری به خود گرفته و امکان استفاده از الگوریتم نمونه‌گیری گیبس فراهم می‌شود. اگرچه افزایش متغیرهای کمکی بعد فضای پارامترها را افزایش می‌دهد، اما با حذف نیاز به گام‌های پذیرش/رد و بهبود اختلاط زنجیره‌ها، کارایی کلی زنجیره را به شدت ارتقا می‌بخشد.

مطالعات متعددی در این زمینه برای مدل‌های انتخاب چندگانه و لجستیک صورت گرفته است. هولمز و هلد (۲۰۰۶) راهبردی مبتنی بر متغیرهای کمکی پیوسته نرمال ارائه کردند که توزیع‌های شرطی پسین ضرایب را به توزیع نرمال چندمتغیره تبدیل می‌کند. فروویرث-اشناتر و فروویرث (۲۰۱۰، ۲۰۱۲) ساختار کاراتری مبتنی بر مدل مطلوبیت تصادفی تفاضلی معرفی نمودند که کارایی محاسباتی بیشتری نسبت به روش هولمز و هلد دارد. ماری (۲۰۲۱) با معرفی متغیرهای کمکی گاما نشان داد که می‌توان وابستگی شدید ضرایب طبقات مختلف در سافت‌مکس را تعدیل کرد و نمونه‌گیری پسین را از طریق الگوریتم‌های انطباقی یا نمونه‌گیری برش بیضوی بهبود بخشید. از سوی دیگر، پولسون و همکاران (۲۰۱۳) با معرفی متغیرهای کمکی با توزیع پلیا–گاما، تحولی در رگرسیون لجستیک بیزی ایجاد کردند؛ زیرا این روش توزیع شرطی ضرایب را به طور کامل به فرم بسته نرمال تبدیل کرده و امکان پیاده‌سازی نمونه‌گیر گیبس واقعی و بدون پذیرش/ رد را فراهم می‌سازد.

در این پژوهش، کارایی محاسباتی و دقت پیش‌بینی چهار الگوریتم نمونه‌گیری پسین در مدل رگرسیون لجستیک چندجمله‌ای مورد مقایسه قرار می‌گیرد. این روش‌ها شامل: الگوریتم متروپولیس–هستینگز تطبیقی ، روش افزایش داده با متغیرهای گاما ترکیب‌شده با متروپولیس تطبیقی، روش افزایش داده گاما با نمونه‌گیری برش بیضوی و در نهایت، روش مبتنی بر افزایش داده پلیا–گاما در چارچوب تجزیه شکستن چوب می‌باشند. این مقایسه‌ها بر روی داده‌های طبقه‌بندی تصاویر ام‌آرآی مغزی انجام شده تا عملکرد هر روش در مواجهه با ویژگی‌های مستخرج از تصویر ($HOG$ و $LBP$کاهش‌یافته با مولفه‌های اصلی ) ارزیابی شود.

## ۲- مدل‌های پژوهش

### ۲-۱ رگرسیون چندجمله‌ای بیزی

رگرسیون لجستیک چندجمله‌ای یکی از تعمیم‌های مدل لجستیک دودویی است که برای مدل‌سازی متغیرهای پاسخ چندکلاسه مورد استفاده قرار می‌گیرد. این مدل در چارچوب مدل‌های لاجیت تعمیم‌یافته توسط فادِن (۱۹۷۴) معرفی شد و به طور گسترده در تحلیل داده‌های طبقه‌بندی چندکلاسه به کار می‌رود. فرض کنید برای هر مشاهده $i \in \{۱, \dots, N\}$ متغیر پاسخ به صورت بردار شاخصی $y_i = (y_{i1}, \dots, y_{iC})$ تعریف شود، به‌گونه‌ای که $y_{ij} = ۱$ اگر مشاهده $i$ در طبقه $j$ قرار گیرد و در غیر این صورت برابر صفر باشد. آنگاه

$$y_i \mid \cdot \sim \text{Multinomial}(n_i, \pi_{i1}, \dots, \pi_{iC})$$

تعریف می‌شود که در آن $n_i$ تعداد کل آزمون‌هادر حالت طبقه‌بندی ساده $n_i = ۱$ و $\pi_{ij}$ احتمال تعلق مشاهده $i$ به طبقه $j$ است، به‌طوریکه $\sum_{j=۱}^{C} \pi_{ij} = ۱$ باشد.

### ۲-۲ مدل لینک لاجیت چندجمله‌ای

اگر $x_i \in R^P$ بردار متغیرهای توضیحی و $\beta_j$ بردار ضرایب مربوط به طبقه j باشد، آنگاه احتمال تعلق مشاهده $i$ به طبقه $j$ از طریق تابع softmax به‌صورت

$$\pi_{ij} = \frac{\exp\left(x_i^T \beta_j\right)}{\sum_{k=1}^{C} \exp(x_i^T \beta_k)}$$

تعریف می‌شود. به دلیل ناوردایی مدل نسبت به انتقال همزمان همه ضرایب، برای شناسایی پارامترها یکی از طبقات به عنوان طبقه مرجع در نظر گرفته می‌شود. در این پژوهش طبقه C به عنوان طبقه مرجع انتخاب شده است. پس می‌بایست $\beta_C = ۰$ باشد و در نتیجه مدل برای $j = ۱, \dots, C - ۱$ به‌صورت $\frac{\boldsymbol{\pi_{ij}}}{\boldsymbol{\pi_{iC}}} = x_i^T \beta_j$ نوشته می‌شود. در چارچوب بیزی، برای ضرایب مدل توزیع پیشین نرمال چندمتغیره $\beta_j \sim \mathcal{N}(\mu_., \Sigma.)$ در نظر گرفته می‌شود؛ در نتیجه، توزیع پسین ضرایب با حاصل‌ضرب درست‌نمایی و پیشین

$$p(\beta j).p\left(\beta_j \mid y\right) \propto \prod_{i=1}^{N} \frac{\exp\left(y_{ij} x_i^T \beta_j\right)}{(\sum_{k=1}^{C} \exp(x_i^T \beta_k))^{n_i}} \, p\left(\beta_j\right).$$

متناسب است. به دلیل وجود جمله جمعی در مخرج تابع سافت‌مکس، توزیع پسین فرم بسته ندارد و امکان استخراج تحلیلی امیدریاضی یا واریانس پارامترها وجود ندارد. بنابراین، برای نمونه‌گیری از توزیع پسین ضرایب مدل، از روش‌های‌نمونه‌گیری زنجیره مارکوف–مونت‌کارلو استفاده می‌شود.

## ۲-۳ افزایش داده با بهره‌گیری از متغیرهای توزیع گاما

یکی از چالش‌های اصلی در رگرسیون لجستیک چندجمله‌ای، وابستگی ضرایب طبقات مختلف از طریق جمله نرمال‌ساز softmax است. این وابستگی باعث می‌شود نمونه‌گیری مستقیم از توزیع پسین پیچیده و از نظر محاسباتی پرهزینه باشد. ایده کلی افزایش داده نخستین بار توسط تانر و ونگ (۱۹۸۷) مطرح شد. در این چارچوب، با معرفی متغیرهای کمکی، ساختار توزیع پسین ساده‌تر شده و نمونه‌گیری کاراتر انجام می‌شود. در این پژوهش، برای هر مشاهده یک متغیر کمکی $\phi_i$ با توزیع گاما

$$\phi_i \mid \cdot \sim \text{Gamma}\left(n_i, \sum_{j=1}^{C} f_j(x_i)\right)$$

معرفی می‌شود؛که در آن $f_j(x_i) = \exp\left(x_i^T \beta_j\right)$ با افزودن این متغیرهای کمکی، توزیع مشترک به‌صورت

$$p(y_i, \phi_i \mid \cdot) = p(\phi_i \mid \cdot)\, p(y_i \mid \cdot)$$

بازنویسی می‌شود؛ که پس از ساده‌سازی به فرم زیر

$$p(y_i, \phi_i \mid \cdot) \propto \prod_{j=1}^{C} f_j(x_i)^{y_{ij}} \exp\left(-\phi_i f_j(x_i)\right)$$

می‌شود. این بازنویسی باعث می‌شود که ضرایب مربوط به هر طبقه به شکل جداگانه‌تر و با وابستگی کمتری نسبت به سایر طبقات نمونه‌گیری شوند. در نتیجه، ساختار شرطی ساده‌تر شده و کارایی الگوریتم‌های MCMC به طور قابل توجهی بهبود می‌یابد.

## ۲-٤ الگوریتم متروپولیس– هاستینگز تطبیقی

الگوریتم متروپولیس- هاستینگز نخستین بار توسط متروپولیس و همکاران (۱۹۵۳) معرفی شد و بعدها توسط هاستینگز(۱۹۷۰) به شکل کلی‌تری تعمیم یافت. این الگوریتم یکی از مهم‌ترین روش‌های نمونه‌گیری در چارچوب روش‌های مونت‌کارلو مبتنی بر زنجیره‌های مارکوف‌است که برای نمونه‌گیری از توزیع‌های پسین پیچیده که فرم بسته ندارند مورد استفاده قرار می‌گیرد. در این روش، در هر تکرار یک مقدار جدید برای پارامترها از یک توزیع پیشنهادی تولید می‌شود و سپس با استفاده از احتمال پذیرش متروپولیس پذیرفته یا رد می‌شود. احتمال پذیرش به‌صورت

$$\alpha = \min\left(1, \frac{p(\beta^*|y)}{p(\beta^{(t)}|y)}\right)$$

تعریف می‌شود؛ که در آن $\beta^*$ مقدار پیشنهادی و $\beta^{(t)}$ مقدار فعلی زنجیره مارکوف است. با وجود سادگی این روش، عملکرد آن به انتخاب مناسب توزیع پیشنهادی وابسته است. اگر ماتریس کوواریانس پیشنهاد به‌درستی انتخاب نشود، زنجیره ممکن است به کندی در فضای پارامترها حرکت کند. برای حل این مشکل، هاریو و همکاران (۲۰۰۱) الگوریتم متروپولیس تطبیقی را معرفی کردند که در آن ماتریس کوواریانس توزیع پیشنهادی در طول اجرای الگوریتم و بر اساس نمونه‌های تولید شده به‌طور خودکار تنظیم می‌شود. این سازوکار تطبیقی باعث می‌شود که الگوریتم به‌تدریج به یک توزیع پیشنهادی مناسب نزدیک شود و در نتیجه اختلاط زنجیره بهبود یابد.

## ۲-۵ افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی

در مدل رگرسیون لجستیک چندجمله‌ای، وجود جمله نرمال‌ساز در تابع softmax باعث می‌شود که ضرایب مربوط به طبقات مختلف به یکدیگر وابسته باشند. این وابستگی نمونه‌گیری از توزیع پسین را پیچیده می‌کند و می‌تواند کارایی الگوریتم‌های MCMC را کاهش دهد. برای کاهش این مشکل، از روش افزایش داده استفاده می‌شود. ایده اصلی افزایش داده در مدل‌های بیزی توسط تانر و ونگ (۱۹۸۷) مطرح شد. بعدها ماری (۲۰۲۱) با استفاده از متغیرهای کمکی گاما، یک چارچوب کارآمد برای مدل‌های لجستیک چندجمله‌ای ارائه کرد که در آن ساختار تابع درست‌نمایی ساده‌تر می‌شود. در این روش، برای هر مشاهده یک متغیر کمکی با توزیع گاما معرفی می‌شود که باعث می‌شود جمله نرمال‌ساز مدل به صورت تجزیه‌پذیر بازنویسی شود. در نتیجه، وابستگی بین ضرایب طبقات کاهش یافته و امکان نمونه‌گیری مستقل‌تر برای پارامترها فراهم می‌شود.

## ۲-٦ افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش بیضوی

افزایش داده گاما با روش نمونه‌گیری برش بیضوی توسط ماری (۲۰۱۰) معرفی شد. این روش نوعی الگوریتم روش نمونه‌گیری برشی Slice Sampling است که به طور خاص برای مدل‌هایی با پیشین نرمال چندمتغیره طراحی شده است.در این الگوریتم، به جای استفاده از یک توزیع پیشنهادی مانند روش متروپولیس–هستینگز، نمونه‌های جدید از طریق حرکت روی یک مسیر بیضوی در فضای پارامترها تولید می‌شوند. این بیضی‌ها توسط ساختار کوواریانس توزیع پیشین نرمال تعیین می‌شوند. یکی از مزایای مهم Elliptical Slice Sampling این است که برخلاف بسیاری از الگوریتم‌های MCMC، نیازی به تنظیم پارامترهایی مانند اندازه گام یا ماتریس کوواریانس پیشنهادی ندارد. به همین دلیل، این روش معمولاً پایداری بیشتری داشته و پیاده‌سازی آن ساده‌تر است. در مدل رگرسیون لجستیک چندجمله‌ای، برای ساده‌سازی ساختار توزیع پسین از افزایش داده مبتنی بر متغیرهای گاما استفاده می‌شود. سپس ضرایب مدل با استفاده از الگوریتم Elliptical Slice Sampling نمونه‌گیری می‌شوند. این ترکیب باعث می‌شود که نمونه‌گیری در فضای پارامترها کاراتر انجام شود و زنجیره‌های مارکوف با سرعت بیشتری به توزیع پایدار برسند.

## ۲-۷ رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما

روش افزایش داده پلیا–گاما نخستین‌بار توسط پولسون و همکاران ( ۲۰۱۳) برای نمونه‌گیری بیزی در مدل‌های لجستیک معرفی شد. ایده اصلی این روش آن است که با وارد کردن متغیرهای کمکی از توزیع پلیا–گاما، درست‌نمایی لجستیک به شکلی بازنویسی می‌شود که توزیع شرطی پسین ضرایب مدل، دارای فرم نرمال چندمتغیره گردد. در نتیجه، نمونه‌گیری از پارامترهای مدل می‌تواند با الگوریتم گیبس انجام شود. در این پژوهش، برای مدل‌سازی پاسخ چندکلاسه، از فرم بیزی رگرسیون لجستیک چندجمله‌ای با افزایش داده پلیا–گاما استفاده می‌شود. از آنجا که پیاده‌سازی مستقیم ساختار چندجمله‌ای در این چارچوب دشوار است، مدل به‌صورت تجزیه stick-breaking نوشته می‌شود و هر مرحله به یک رگرسیون لجستیک دودویی بیزی تبدیل می‌گردد. فرض کنید برای مشاهده $i$ متغیر پاسخ

$$Y_i \in \{۱,۲, \dots, C\}$$

و بردار ویژگی‌ها برابر با $x_i \in R^P$ باشد. در نمایش stick-breaking، احتمال تعلق مشاهده $i$ به کلاس $j$ به‌صورت

$$\pi_{i1} = \sigma(x_i^{\top}\beta_۱)$$

$$\pi_{ij} = \sigma\left(x_i^\top \beta_j\right) \prod_{k=۱}^{j-۱} [۱ - \sigma(x_i^\top \beta_k)], \qquad j = ۲, \dots, C-۱$$

$$\pi_{iC} = \prod_{k=۱}^{C-۱} [۱ - \sigma(x_i^\top \beta_k)]$$

تعریف می‌شود؛ که در آن

$$\sigma(t) = \frac{۱}{۱ + e^{-t}}$$

تابع لجستیک و $\beta_j$ بردار ضرایب مربوط به مرحله $j$ است. بنابراین:

$$Y_i \sim \text{Categorical}(\pi_{i1}, \pi_{i2}, \dots, \pi_{iC})$$

برای هر مرحله $j$، با تعریف پاسخ دودویی مناسب، مدل شرطی به‌صورت

$$z_{ij} \mid \beta_j \sim \text{Bernoulli}\left(\sigma\left(x_i^\top \beta_j\right)\right)$$

نوشته می‌شود.برای ضرایب هر مرحله نیز پیشین نرمال

$$\beta_j \sim \mathcal{N}(\mu_\cdot, \Sigma_\cdot), \qquad j = ۱, \dots, C-۱.$$

در نظر گرفته می‌شود. در روش پلیا–گاما، برای هر مشاهده یک متغیر کمکی به‌صورت

$$\omega_{ij} \sim PG\left(۱, x_i^\top \beta_j\right)$$

معرفی می‌شود. با استفاده از این متغیر کمکی، توزیع شرطی پسین ضرایب به فرم نرمال چندمتغیره به‌صورت

$$\beta_j \mid \omega_j, z_j \sim \mathcal{N}\left(m_j, V_j\right)$$

تبدیل می‌شود. که در آن

$$V_j = \left(X_j^\top \Omega_j X_j + \Sigma_\cdot^{-۱}\right)^{-۱}$$

$$m_j = V_j\left(X_j^\top \kappa_j + \Sigma_\cdot^{-۱} \mu_\cdot\right)$$

و در آن

$$\Omega_j = \mathrm{diag}\left(\omega_{1j}, \dots, \omega_{n_j j}\right), \qquad \kappa_j = z_j - \frac{1}{2}$$

بنابراین، در هر مرحله از الگوریتم، نمونه‌گیری به‌صورت گیبس و از دو توزیع شرطی زیر

$$\omega_{ij} \mid \beta_j \sim PG\left(1, x_i^\top \beta_j\right)$$

$$\beta_j \mid \omega_j, z_j \sim \mathcal{N}\left(m_j, V_j\right)$$

انجام می‌شود. در این پژوهش، کلاس مرجع به‌عنوان آخرین کلاس در نظر گرفته شده و مدل پلیا–گاما به‌صورت stick-breaking چندکلاسه برای برآورد ضرایب و پیش‌بینی کلاس نهایی به کار رفته است.

## ۳- معیارهای ارزیابی عملکرد مدل

### ۳-۱ اندازه نمونه مؤثر

اندازه نمونه مؤثر یکی از مهم‌ترین شاخص‌ها برای سنجش کیفیت نمونه‌های تولیدشده توسط زنجیره مارکوف است. در یک حالت ایده‌آل، اگر نمونه‌ها مستقل باشند، حجم نمونه واقعی برابر تعداد کل نمونه‌هاست. اما در الگوریتم‌های MCMC، نمونه‌ها معمولاً به یکدیگر وابسته‌اند و این وابستگی باعث کاهش مؤثر اطلاعات موجود در نمونه‌ها می‌شود. معیار ESS نشان می‌دهد که مجموعه نمونه‌های وابسته به‌دست‌آمده، از نظر میزان اطلاعات، معادل چه تعداد نمونه‌ی مستقل و هم‌توزیع است. فرض کنید M تعداد کل نمونه‌های استخراج شده باشد؛ مقدار ESS طبق فرمول به‌صورت

$$ESS = \frac{M}{1 + 2\sum_{k=1}^{\infty} \rho_k}$$

محاسبه می‌شود؛ که در آن $\rho_k$ نشان‌دهنده خودهمبستگی زنجیره در تاخیر k است. هرچه مقدار ESS بزرگ‌تر باشد، کیفیت اختلاط زنجیره بهتر بوده و تخمین پارامترهای پسین با دقت بیشتری انجام می‌شود. مفهوم اولیه اندازه نمونه مؤثر در ادبیات آمار بیزی به بحث‌های واریانس اصلاح‌شده در حضور خودهمبستگی بازمی‌گردد. استفاده از این معیار در تحلیل زنجیره مارکوف ابتدا در آثار کاس، کارلین، گلمان و نیل (۱۹۹۸) مطرح شد و سپس رابرت و کاسلا (۲۰۰۴) به صورت رسمی و استاندارد در ارزیابی الگوریتم‌های MCMC توسعه دادند.

### ۲-۳ نرخ نمونه مؤثر

در کنار معیار ESS که کیفیت نمونه‌ها را می‌سنجد، لازم است کارایی زمانی الگوریتم نیز در نظر گرفته شود. ممکن است الگوریتمی ESS بالایی تولید کند، اما هزینه محاسباتی آن چنان زیاد باشد که در واحد زمان، اطلاعات بسیار اندکی استخراج شود. برای ترکیب این دو جنبه – کیفیت نمونه‌ها و سرعت اجرای الگوریتم – معیار نرخ نمونه مؤثر استفاده می‌شود. این معیار به صورت نسبت اندازه نمونه مؤثر به زمان کل اجرای الگوریتم بر حسب ثانیه به‌صورت

$$ESR = \frac{ESS}{\text{Runtime (s)}}$$

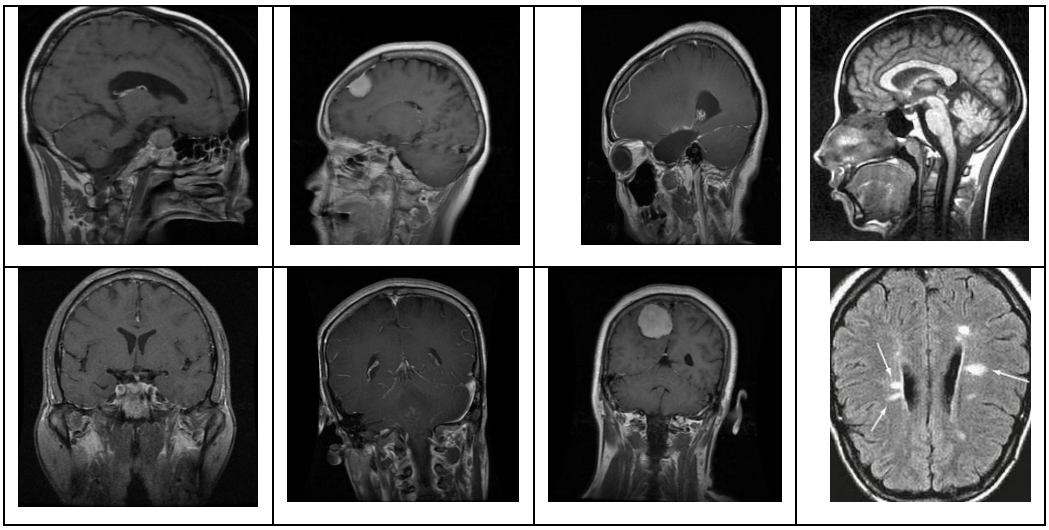
تعریف می‌گردد. بنابراین، ESR بیانگر مقدار اطلاعات مفید استخراج‌شده از زنجیره به‌ازای هر ثانیه محاسبه است و الگوریتمی که ESR بالاتری تولید کند، در توازن میان دقت و سرعت کاراتر است. ایده استفاده از ESS در مقایسه الگوریتم‌های بیزی نخستین‌بار در کارهای محاسباتی مربوط به الگوریتم‌های هامیلتونی توسط جیرولامی و کالدرهد (۲۰۱۱) مطرح شد. این معیار بعدها در توسعه زبان Stan توسط کارپنتر و همکاران (۲۰۱۷) به‌عنوان معیار اصلی مقایسه میان روش‌های نمونه‌گیری مختلف، الگوریتم‌های مبتنی بر متروپولیس، تثبیت شد. در این پژوهش نیز، ESR به‌عنوان معیار اصلی برای انتخاب الگوریتم کاراتر مورد استفاده قرار گرفته است؛ زیرا مقایسه صرف ESS بدون درنظرگرفتن زمان اجرا می‌تواند انتخاب الگوریتم را دچار خطا کند.

## ٤ - تحلیل داده‌های تصاویر تومور مغزی

در این پژوهش از مجموعه‌داده MRI شامل چهار کلاس توموری و غیرتوموری شامل glioma، meningioma، notumor و pituitary استفاده شد. تصاویر از مسیر

https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset?resource=download

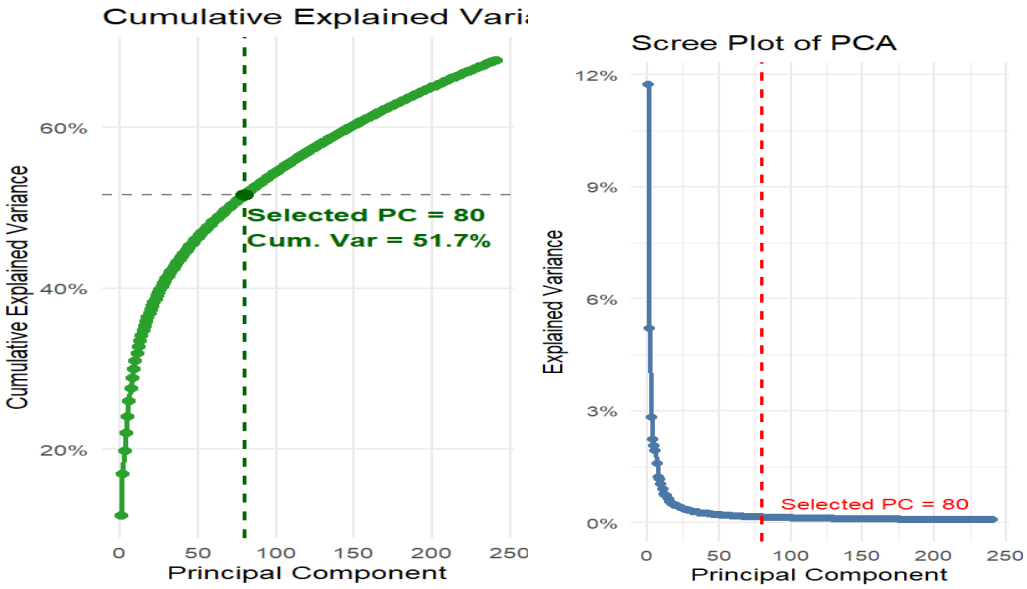
بارگذاری شدند. برای هر کلاس ۱۲۰ تصویر برای آموزش و ۴۰ تصویر برای آزمون انتخاب گردید. نمونه‌ای از تصاویر به صورت زیر ارائه شده است.



جدول۱- توزیع تعداد نمونه‌ها بر اساس کلاس و مجموعه داده

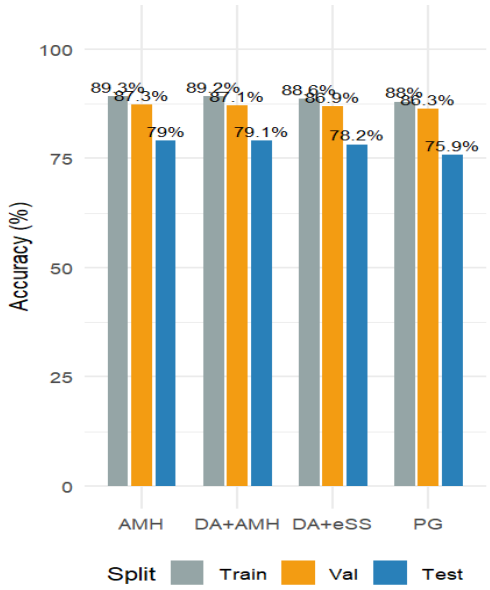
| کلاس | (Train) آموزش | (Val) اعتبارسنجی | (Test) آزمون |
|---|---|---|---|
| Glioma | ۱۰۵۰ | ۳۵۰ | ۴۰۰ |
| Meningioma | ۱۰۵۰ | ۳۵۰ | ۴۰۰ |
| Notumor | ۱۰۴۷ | ۳۴۹ | ۴۰۰ |
| Pituitary | ۱۰۵۰ | ۳۵۰ | ۴۰۰ |
| جمع کل | ۴۱۹۷ | ۱۳۹۹ | ۱۶۰۰ |

بر اساس جدول۱، توزیع نمونه‌ها میان چهار کلاس هدف از توازن بالایی برخوردار است؛ در مجموع، برای هر یک از کلاس‌های Glioma، Meningiomaو Pituitary، تعداد ۱۰۵۰ نمونه در مجموعه آموزش، ۳۵۰ نمونه در مجموعه اعتبارسنجی و ۴۰۰ نمونه در مجموعه آزمون در نظر گرفته شده است. در مورد کلاس NoTumor، به دلیل بروز خطای پردازشی در استخراج ویژگی برای ۴ تصویر، تعداد نمونه‌ها با اختلاف بسیار اندک نسبت به سایر کلاس‌ها، شامل ۱۰۴۷ نمونه در بخش آموزش، ۳۴۹ نمونه در بخش اعتبارسنجی و ۴۰۰ نمونه در بخش آزمون است. این توزیع تقریباً یکنواخت در هر سه زیرمجموعه، تضمین می‌کند که مدل در مراحل یادگیری و ارزیابی، با سوگیری نسبت به کلاس خاصی مواجه نشود و معیار دقت بازتاب‌دهنده عملکرد واقعی مدل باشد.



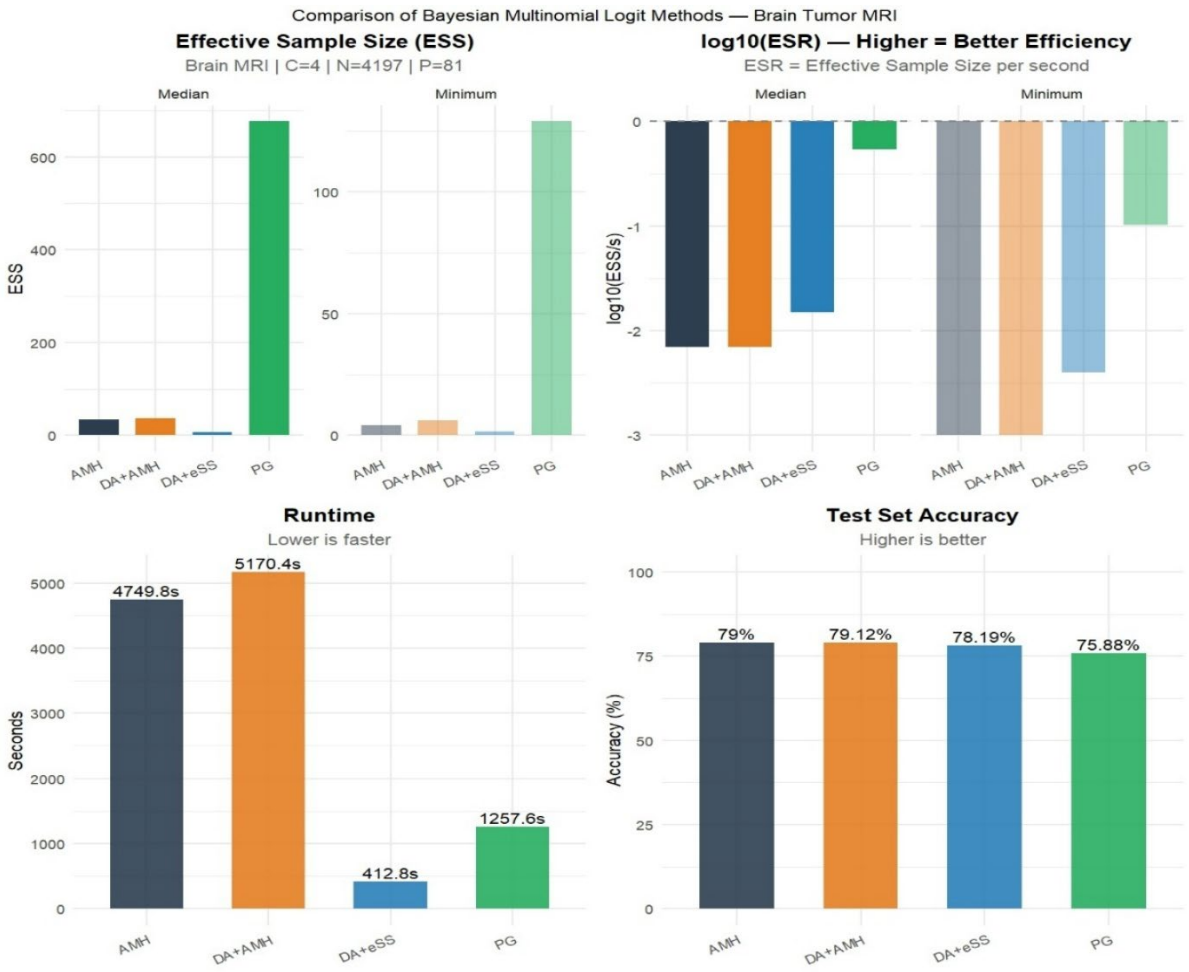

شکل ۱- نمودار اسکری (سمت چپ) و نمودار واریانس تجمعی(سمت راست)

نمودار اسکری (شکل سمت چپ) خیلی خوب نشان می‌دهد که واریانس داده‌ها چطور در مؤلفه‌های اصلی پخش شده است. همان‌طور که می‌بینید، در همان ۱۰ مؤلفه اول یک افت ناگهانی و شدید داریم؛ این یعنی بیشتر اطلاعات مهم و تغییرات اصلی داده‌ها در همین مؤلفه‌های ابتدایی خلاصه شده است. بعد از آن، منحنی خیلی زود شیبش را از دست می‌دهد و به شکل یک دنباله بلند درمی‌آید که نشان می‌دهد مؤلفه‌های بعدی تغییرات خیلی جزئی را در بر می‌گیرند. این رفتار در ویژگی‌های تصویری مثل HOG و LBP کاملاً طبیعی است، چون پیکسل‌های مجاور همبستگی بالایی دارند و اطلاعات هم‌پوشان زیادی تولید می‌کنند. همچنین عدد ۸۰ را به عنوان نقطه قطع انتخاب کردیم. اگر به نمودار دقت کنید، این نقطه دقیقاً همان‌جایی است که منحنی وارد فاز تخت شدن یا نقطه Elbow می‌شود. یعنی از اینجا به بعد، اضافه کردن مؤلفه‌های بیشتر، عملاً اطلاعات جدید قابل‌توجهی به ما نمی‌دهد و فقط بار محاسباتی را الکی سنگین می‌کند. در نمودار دوم (سمت راست)، روند تغییرات واریانس تجمعی نشان می‌دهد که با انتخاب ۸۰ مؤلفه اول، در مجموع حدود ٪۵۱.۷ از واریانس کل داده‌ها حفظ شده است. هرچند این عدد در نگاه اول کمی پایین به نظر می‌رسد، اما این امر ناشی از غیرخطی و پیچیده بودن فضای ویژگی‌های ترکیبی HOG و LBP است که روش خطی PCA نمی‌تواند تمام پیچیدگی‌های آن را در ابعاد پایین بازسازی کند. از طرفی، انتخاب این تعداد مؤلفه حاصل یک موازنه (Trade-off) آگاهانه میان پایداری محاسباتی و دقت مدل بوده است؛ چرا که در نمونه‌گیری‌های بیزی و زنجیره‌های MCMC، افزایش ابعاد ویژگی‌ها برای کسب واریانس بیشتر، به دلیل پدیده بعدزدگی مانع از همگرایی زنجیره‌ها می‌شد. در نتیجه، انتخاب ۸۰ مؤلفه به عنوان یک نقطه بهینه، علاوه بر تضمین همگرایی و کارایی محاسباتی مدل، بستر مناسبی برای دستیابی به دقت پایدار ۷۵ تا ۷۹ درصدی فراهم کرده است.




شکل۲- مقایسه‌ای عملکرد مدل‌ها بر اساس معیارهای دقت

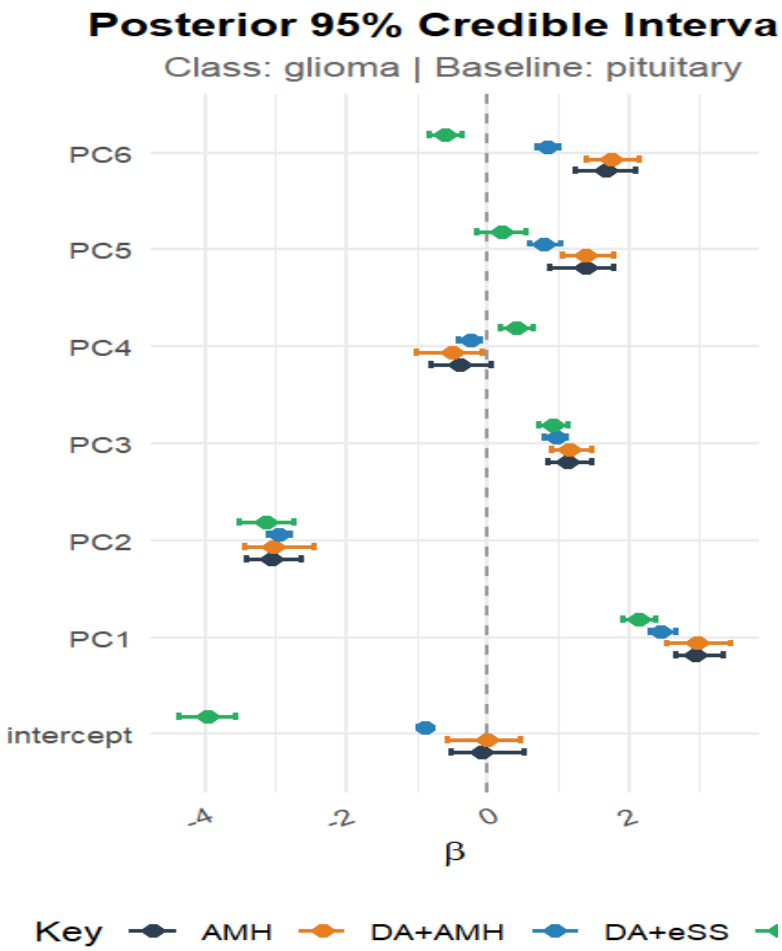
همان‌طور که در شکل۲ دیده می‌شود، نشان‌دهنده الگوی رفتاری مشابهی در تمامی مدل‌ها است؛ به‌طوری‌که دقت روی داده‌های آموزش در محدوده‌ی ۸۸٪ تا ۸۹.۳٪، روی داده‌های اعتبارسنجی بین ۸۶.۳٪ تا ۸۷.۳٪، و روی داده‌های آزمون بین ۷۵.۹٪ تا ۷۹.۱٪ قرار دارد. در این میان، روش افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی با کسب دقت آزمون ۷۹.۱٪، بهینه‌ترین عملکرد را در پیش‌بینی داده‌های نادیده از خود نشان داده است و پس از آن، روش کلاسیک الگوریتم متروپولیس– هاستینگز تطبیقی با اختلاف اندک (۷۹.۰٪ دقت آزمون) در رتبه دوم قرار دارد. در مقابل، روش نمونه‌گیری پولیا-گاما علی‌رغم ویژگی‌های برجسته در سرعت و کارایی محاسباتی، با ثبت دقت آزمون ۷۵.۹٪، ضعیف‌ترین عملکرد را در این ارزیابی ثبت کرده است.تفاوت عملکرد مدل‌ها در داده‌های آموزش و آزمون، ریشه در حذف حدود ۴۸.۳٪ از واریانس کل داده‌ها در مرحله کاهش بعد با تحلیل مؤلفه‌های اصلی دارد. این موازنه آگاهانه، اگرچه کارایی محاسباتی و همگرایی زنجیره‌های MCMC را تضمین کرده، اما به قیمت از دست رفتن برخی ویژگی‌های جزئی تصویر تمام شده است. با این حال، برتری عملکرد روش ترکیبی روش افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی نشان می‌دهد که ادغام تکنیک افزونگی داده با نمونه‌گیرهای انطباقی، پایداری زنجیره را بهبود بخشیده و با تقریب دقیق‌تر توزیع پسین، مدل را در مواجهه با داده‌های جدید توانمندتر ساخته است.




شکل۳- نمودار ارزیابی استقلال آماری و کیفیت تولید نمونه‌های غیرهمبسته بر اساس شاخص ESS (سمت بالا-چپ) و نمودار بهره‌وری الگوریتم به ازای واحد زمان در مقیاس لگاریتمی ESR( سمت بالا- راست) و نمودار سنجش بار محاسباتی و زمان کل شبیه‌سازی زنجیره‌های مارکوف ( سمت پایین-چپ) و نمودار ارزیابی قدرت تفکیک و دقت نهایی مدل بر روی داده‌های آزمون (سمت پایین – راست)

مطابق با نتایج ارائه شده در نمودار ارزیابی استقلال آماری و کیفیت تولید نمونه‌های غیرهمبسته بر اساس شاخص ESS (سمت بالا-چپ تفاوت ساختاری عمیقی در کیفیت نمونه‌گیری روش‌ها مشاهده می‌شود. روش رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما در هر دو شاخص میانه و حداقل، با اختلافی فاحش نسبت به سایر رقیبان، بیشترین میزان نمونه‌های مستقل را تولید کرده است. این مقدار بالای ESS در روش و رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما نشان‌دهنده توانمندی این الگوریتم در شکستن خودهمبستگی درون زنجیره و پیمایش سریع فضای پارامترهاست. در مقابل، مقادیر پایین این شاخص در روش‌های خانواده الگوریتم متروپولیس– هاستینگز تطبیقی حاکی از آن است که زنجیره در این مدل‌ها با کندی حرکت کرده و نمونه‌ها وابستگی شدیدی به یکدیگر دارند. همچنین در نمودار بهره‌وری الگوریتم به ازای واحد زمان در

مقیاس لگاریتمی ESR( سمت بالا- راست) ترسیم شده است، روش روش و رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما با نزدیک شدن به سطح صفر در مقیاس لگاریتمی، بهینه‌ترین عملکرد را در تولید نمونه مستقل به ازای هر ثانیه از زمان محاسبات نشان می‌دهد. این نمودار به وضوح تایید می‌کند که روش‌های الگوریتم متروپولیس– هاستینگز تطبیقی و افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس–هاستینگز تطبیقی، به دلیل ترکیب هم‌زمان همبستگی بالا و سرعت پایین، در پایین‌ترین سطح بهره‌وری قرار دارند، در حالی که روش افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش بیضوی توانسته است جایگاه خود را به عنوان یک روش کارآمد تثبیت کند. همچنین در نمودار سنجش بار محاسباتی و زمان کل شبیه‌سازی زنجیره‌های مارکوف ( سمت پایین-چپ) تفاوت در هزینه‌های پردازشی مدل‌ها کاملاً مشهود است. روش افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش بیضوی با ثبت زمان خیره‌کننده ۴۱۲.۸ ثانیه، سریع‌ترین عملکرد را در همگرایی زنجیره داشته است، در حالی که روش افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی، با صرف ۵۱۷۰.۴ ثانیه (تقریباً ۱۲ برابر زمان بیشتر)، سنگین‌ترین بار محاسباتی را به سیستم تحمیل کرده است. این شکاف زمانی نشان می‌دهد که استفاده از نمونه‌گیرهای انطباقی اگرچه دقیق است، اما در پروژه‌های بزرگ با محدودیت‌های زمانی یک چالش جدی به شمار می‌آید. در نمودار ارزیابی قدرت تفکیک و دقت نهایی مدل بر روی داده‌های آزمون (سمت پایین – راست) پایین-راست نشان‌دهنده پارادوکس میان کارایی محاسباتی و دقت پیش‌بینی است. برخلاف نتایج بخش‌های قبلی، روش افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی با کسب دقت ۷۹.۱۲٪، برترین عملکرد را در طبقه‌بندی تصاویر نادیده (تست) از خود نشان داده است. روش PG که در تمام شاخص‌های محاسباتی پیشتاز بود، در این مرحله با دقت ۷۵.۸۸٪ در جایگاه آخر قرار گرفته است. این موضوع ثابت می‌کند که دقت بالاتر در تخمین پارامترهای تصاویر پیچیده MRI، لزوماً با سرعت نمونه‌گیری هم‌سو نیست و نیاز به کاوش دقیق‌تر در فضای پسین دارد.




شکل۴- ارزیابی عملکرد روش‌های نمونه‌گیری در مدل بیزی

در شکل۴ مشاهده می‌شود، مقایسه‌ی چهار روش روی داده‌های MRI مغز، یک الگوی روشن از trade-off بین سرعت، کیفیت نمونه‌گیری و دقت دیده می‌شود. روش رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما از نظر ESS و ESR بهترین عملکرد را دارد و نشان می‌دهد نمونه‌های تولیدشده از نظر آماری کیفیت بالایی دارند، اما با وجود این برتری، زمان اجرای آن حدود ۲۱ دقیقه است و دقت آزمونش هم پایین‌ترین مقدار بین روش‌ها یعنی حدود ۷۵.۹٪ بوده است. در مقابل، افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش بیضوی سریع‌ترین روش است و فقط حدود ۷ دقیقه زمان می‌برد، اما کیفیت نمونه‌گیری آن ضعیف‌تر از بقیه است؛ با این حال دقت آن حدود ۷۸.۲٪ باقی مانده و همین باعث می‌شود از نظر عملی گزینه‌ی متعادل‌تری باشد. از طرف دیگر، الگوریتم متروپولیس– هاستینگز تطبیقی و افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی کندترین روش‌ها هستند و زمان اجرای آن‌ها به‌ترتیب بیش از ۷۹ و ۸۶ دقیقه است، بدون اینکه در ازای این هزینه‌ی زمانی، برتری خیلی چشمگیری در کیفیت نمونه‌گیری یا دقت نهایی به‌دست آورند؛ هرچند افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی با دقت

حدود ۷۹.۱٪ بهترین نتیجه را ثبت کرده، اما اختلاف آن با سایر روش‌ها خیلی زیاد نیست. بنابراین، اگر بخواهیم بر اساس این نتایج جمع‌بندی کنیم، افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش بیضوی به‌خاطر تعادل خوب بین سرعت و دقت، و رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما به‌خاطر کیفیت نمونه‌گیری بالا اما نه دقت بهتر، دو روش برجسته‌تر هستند و الگوریتم متروپولیس– هاستینگز تطبیقی و افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی بیشتر در دسته‌ی روش‌های پرهزینه از نظر زمان قرار می‌گیرند.

جدول ۲- ماتریس درهم‌ریختگی روش افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی

| پیش‌بینی / واقعی | گلیوما | منتژیوما | بدون تومور | هیپوفیز |
|---|---|---|---|---|
| گلیوما | ۲۳۸ | ۴۳ | ۰ | ۵ |
| منتژیوما | ۸۷ | ۲۶۲ | ۴ | ۲۲ |
| بدون تومور | ۶۰ | ۶۷ | ۳۹۵ | ۲ |
| هیپوفیز | ۱۵ | ۲۸ | ۱ | ۳۷۱ |

با توجه به اینکه روش افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی بالاترین دقت را روی داده‌های آزمون ثبت کرده است، ماتریس درهم‌ریختگی این مدل روی داده‌های آزمون در جدول ۲ مورد تحلیل قرار گرفت. نتایج نشان می‌دهد که مدل در تشخیص کلاس No Tumor با صحت ۹۸.۷۵٪ (شناسایی ۳۹۵ نمونه از ۴۰۰ نمونه) فوق‌العاده دقیق عمل می‌کند که به دلیل تفاوت‌های بارز بافت نرمال با بافت‌های تومورال در تصاویر MRI است.بیشترین خطای طبقه‌بندی مربوط به تداخل دو کلاس Glioma و Meningiomaاست؛ به‌طوری‌که ۸۷ نمونه از تومورهای Meningiomaبه اشتباه گلیوما تشخیص داده شده‌اند و ۴۳ نمونه گلیوما نیز در کلاس منتژیوما قرار گرفته‌اند. این تداخل بالا را می‌توان به شباهت‌های بصری و ساختاری شدید این دو نوع تومور در تصاویر تشدید مغناطیسی نسبت داد که تفکیک آن‌ها را برای ویژگی‌های بافتی سطح پایین نظیر LBP/HOG دشوار می‌سازد. همچنین، تومورهای کلاسPituitary نیز عمدتاً با منتژیوما اشتباه گرفته شده‌اند که نشان‌دهنده‌ی مرزهای تصمیم‌گیری مشترک در فضای ویژگی فشرده‌شده است.

## ۵- نتیجه‌گیری

نتایج این مطالعه نشان می‌دهد که در رگرسیون لجستیک چندکلاسه، نوع نمونه‌گیر بیزی هم بر کارایی زنجیره‌های MCMC و هم بر عملکرد طبقه‌بندی اثر می‌گذارد. از میان روش‌های بررسی‌شده، رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما از نظر همگرایی و کارایی محاسباتی بهترین عملکرد را داشته است؛ به‌طوری‌که کمترین خودهمبستگی، بیشترین اندازه‌ی نمونه‌ی مؤثر و مناسب‌ترین شاخص‌های همگرایی را در مقایسه با سایر روش‌ها نشان می‌دهد. با این حال، این برتری در سطح MCMC به معنای بهترین دقت پیش‌بینی نیست، زیرا در ارزیابی نهایی روی داده‌ی آزمون، افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی با دقت ٪۷۹.۱۲بهترین نتیجه را به‌دست آورده و پس از آن الگوریتم متروپولیس– هاستینگز تطبیقی با ٪۷۹.۰۰قرار گرفته است. روش افزایش داده مبتنی بر متغیرهای گاما با روش نمونه‌گیری برش اگرچه از نظر زمان اجرا سریع‌تر از الگوریتم متروپولیس– هاستینگز تطبیقی و افزایش داده مبتنی بر متغیرهای گاما با الگوریتم متروپولیس– هاستینگز تطبیقی بوده، اما در دقت نهایی و کیفیت همگرایی برتری محسوسی نشان نداده است. در مجموع، نتایج بیانگر آن است که میان کارایی نمونه‌گیری بیزی و دقت طبقه‌بندی یک موازنه وجود دارد؛ به‌این معنا که روش‌هایی با همگرایی بهتر الزاماً بهترین دقت پیش‌بینی را تضمین نمی‌کنند و انتخاب روش نهایی باید بر اساس هدف تحلیل انجام شود.

### پیشنهادات:

با توجه به نتایج این تحقیق، برای کارهای بعدی پیشنهاد می‌شود که در مدل‌سازی رگرسیون لجستیک بیزی چندکلاسه، از روش رگرسیون لجستیک چندجمله‌ای بیزی با افزایش داده پلیا–گاما به عنوان الگوریتم اصلی نمونه‌گیری استفاده کنیم؛ چون این روش سرعت همگرایی و نرخ نمونه‌برداری مؤثر (ESS) بسیار بهتری دارد. با این کار، در منابع محاسباتی صرفه‌جویی می‌شود. از طرف دیگر، از آنجا که ویژگی‌های دستی مثل HOG **و** LBP اطلاعات محدودی از داده‌ها را پوشش می‌دهند و باعث می‌شوند مدل در تشخیص تومورهای شبیه به هم (مثل گلیوما و مننژیوما) دچار خطا شود، بهتر است در آینده از ویژگی‌های استخراج‌شده توسط شبکه‌های عصبی کانولوشنی (CNN) استفاده کنیم تا فاصله بین دقت آموزش و تست کمتر شود. در نهایت، انجام تحلیل حساسیت روی توزیع‌های پیشین (Priors) و تست مدل با مقادیر اولیه مختلف، به ما کمک می‌کند تا مطمئن شویم نتایج مدل پایدار و قابل تعمیم هستند.

منابع:

Carpenter, B. Gelman, A. Hoffman, M. D. Lee, D. Goodrich, B. Betancourt, M. Brubaker, M. Guo, J. Li, P. & Riddell, A. (۲۰۱۷). Stan: A probabilistic programming language. Journal of Statistical Software, ۷٦(۱), ۱–۳۲.

Gelman, A. Carlin, J. B. Stern, H. S., Dunson, D. Vehtari, A., & Rubin, D. B. (۲۰۱۳). Bayesian Data Analysis (۳rd ed.). Chapman & Hall/CRC.

Girolami, M. & Calderhead, B. (۲۰۱۱). Riemann manifold Langevin and Hamiltonian Monte Carlo methods. Journal of the Royal Statistical Society: Series B (Statistical Methodology), ۷۳(۲), ۱۲۳–۲۱٤.

Haario, H. Saksman, E. & Tamminen, J. (۲۰۰۱). An adaptive Metropolis algorithm. Bernoulli, ۷(۲), ۲۲۳–۲٤۲.

Hastings, W. K. (۱۹۷۰). Monte Carlo sampling methods using Markov chains and their applications. Biometrika, ٥۷(۱), ۹۷–۱۰۹.

Kass, R. E. Carlin, B. P. Gelman, A. & Neal, R. M. (۱۹۹۸). Markov Chain Monte Carlo in practice: A roundtable discussion. The American Statistician, ٥۲(۲), ۹۳–۱۰۰.

Martin, A. D. & Quinn, K. M. (۲۰۰٦). MCMCpack: Markov chain Monte Carlo in R. Journal of Statistical Software, ٤۲(۹), ۱–۲۱.

McFadden, D. (۱۹۷٤). Conditional logit analysis of qualitative choice behavior. Frontiers in Econometrics, ۱۰٥–۱٤۲.

Metropolis, N. Rosenbluth, A, Rosenbluth, M. Teller, A. & Teller, E. (۱۹٥۳). Equation of state calculations by fast computing machines. Journal of Chemical Physics, ۲۱(٦), ۱۰۸۷–۱۰۹۲.

Murray, I. Adams, R. & MacKay, D. (۲۰۱۰). Elliptical slice sampling. Proceedings of the ۱۳th International Conference on Artificial Intelligence and Statistics, ٥٤۱–٥٤۸.

Murray, J. S. (۲۰۲۱). Log-linear Bayesian additive regression trees for multinomial logistic models. Journal of Computational and Graphical Statistics.

Polson, N. G. Scott, J. G. & Windle, J. (٢٠١٣). Bayesian inference for logistic models using Pólya–Gamma latent variables. Journal of the American Statistical Association, ١٠٨(٥٠٤), ١٣٣٩–١٣٤٩.

Robert, C. P. & Casella, G. (٢٠٠٤). Monte Carlo Statistical Methods (٢nd ed.). Springer.

Tanner, M. A. & Wong, W. H. (١٩٨٧). The calculation of posterior distributions by data augmentation. Journal of the American Statistical Association, ٨٢(٣٩٨), ٥٢٨–٥٤٠.

پیوست:

(برای سادگی می‌توان در گیت‌هاب قرار داد ولی برای دسترسی راحت‌تر در این پروژه این قسمت قرار دادم. )

```
rm(list = ls(all = TRUE))
gc(FALSE)
library(BayesMultiLogit)
library(jpeg)
library(png)
library(coda)
library(ggplot۲)
library(dplyr)
library(tidyr)
library(gridExtra)
library(caret)
library(mvtnorm)
library(BayesLogit)
library(scales)
library(nnet)
set.seed(١٤٠٢)


DATA_ROOT  <- "D:/Arshad/trm۲_beheshti/bayes/nahaei_b/archive"
```

```
CLASS_NAMES <- c("glioma", "meningioma", "notumor", "pituitary")
C <- length(CLASS_NAMES)

N_PER_CLASS  <- Inf
N_TEST_CLASS <- Inf
VAL_FRAC <- ۰,۲۵

# Feature Extraction

read_gray <- function(path) {
  ext <- tolower(tools::file_ext(path))
  img <- tryCatch({
    if (ext == "jpg" || ext == "jpeg") readJPEG(path)
    else if (ext == "png")           readPNG(path)
    else                             stop("فرمت ناشناخته")
  }, error = function(e) NULL)
  if (is.null(img)) return(NULL)
  if (length(dim(img)) == ۳) {
    gray <- ۰,۲۹۹ * img[,,۱] + ۰,۵۸۷ * img[,,۲] + ۰,۱۱٤ * img[,,۳]
  } else {
    gray <- img
  }
  gray
}
```

```
resize_mat <- function(mat, target = ٩٦L) {
  nr <- nrow(mat); nc <- ncol(mat)
  row_idx <- round(seq(١, nr, length.out = target))
  col_idx <- round(seq(١, nc, length.out = target))
  mat[row_idx, col_idx]
}

hog_features <- function(gray, cell_size = ٨L, n_bins = ٩L) {
  sz  <- nrow(gray)
  n_cells <- sz %/% cell_size

  Gx <- cbind(gray[, -١], gray[, ncol(gray)]) -
    cbind(gray[, ١],  gray[, -ncol(gray)])
  Gy <- rbind(gray[-١, ], gray[nrow(gray), ]) -
    rbind(gray[١, ],   gray[-nrow(gray), ])

  mag   <- sqrt(Gx^٢ + Gy^٢)
  angle <- (atan٢(Gy, Gx) * ١٨٠ / pi) %% ١٨٠

  feats <- c()
  for (i in seq_len(n_cells)) {
    for (j in seq_len(n_cells)) {
      r  <- ((i-١)*cell_size + ١):(i*cell_size)
```

```
    cc <- ((j-١)*cell_size + ١):(j*cell_size)
    r  <- r[r <= sz]; cc <- cc[cc <= sz]
    cell_mag <- mag[r, cc]
    cell_ang <- angle[r, cc]
    hist_vals <- numeric(n_bins)
    bin_width <- ١٨٠ / n_bins
    for (b in seq_len(n_bins)) {
      lo <- (b-١)*bin_width; hi <- b*bin_width
      mask <- cell_ang >= lo & cell_ang < hi
      hist_vals[b] <- sum(cell_mag[mask])
    }
    norm <- sqrt(sum(hist_vals^٢) + ١e-٦)
    feats <- c(feats, hist_vals / norm)
   }
  }
  feats
}

# LBP کد در ٢٥۶ با تجمیع صحیح n_bins گروه
lbp_features <- function(gray, n_bins = ١٦L) {
  nr <- nrow(gray); nc <- ncol(gray)
  lbp_img <- matrix(٠L, nr, nc)
  offsets <- list(c(-١,-١),c(-١,٠),c(-١,١),c(٠,١),
                  c(١,١),c(١,٠),c(١,-١),c(٠,-١))
```

```
  for (k in seq_along(offsets)) {
    dr <- offsets[[k]][١]; dc <- offsets[[k]][٢]
    r_s <- pmax(١, ١+dr); r_e <- pmin(nr, nr+dr)
    c_s <- pmax(١, ١+dc); c_e <- pmin(nc, nc+dc)
    r_t <- pmax(١, ١-dr); r_te <- pmin(nr, nr-dr)
    c_t <- pmax(١, ١-dc); c_te <- pmin(nc, nc-dc)
    neighbor <- matrix(٠, nr, nc)
    neighbor[r_t:r_te, c_t:c_te] <- gray[r_s:r_e, c_s:c_e]
    lbp_img <- lbp_img + (neighbor >= gray) * (٢L^(k-١L))
  }
  full_hist <- tabulate(lbp_img + ١L, nbins = ٢٥٦L)
  group_size <- ٢٥٦L %/% n_bins
  h <- vapply(seq_len(n_bins), function(b) {
    lo <- (b - ١L) * group_size + ١L
    hi <- if (b == n_bins) ٢٥٦L else b * group_size
    sum(full_hist[lo:hi])
  }, numeric(١))
  h / (sum(h) + ١e-٩)
}

intensity_stats <- function(gray) {
  v <- as.vector(gray)
  c(mean   = mean(v),
    sd     = sd(v),
```

```
    q٢٥    = quantile(v, ٠,٢٥, names = FALSE),
    q٧٥    = quantile(v, ٠,٧٥, names = FALSE),
    q٩٠    = quantile(v, ٠,٩٠, names = FALSE),
    contrast = max(v) - min(v),
    energy   = mean(v^٢),
    entropy  = {
      h <- hist(v, breaks = ٣٢, plot = FALSE)$counts
      p <- h / sum(h); p <- p[p > ٠]
      -sum(p * log(p))
    })
}

extract_one <- function(path, img_size = ٩٦L) {
  gray <- read_gray(path)
  if (is.null(gray)) return(NULL)
  gray <- resize_mat(gray, img_size)
  gray <- (gray - min(gray)) / (max(gray) - min(gray) + ١e-٩)

  c(
    hog_features(gray, cell_size = ٨L, n_bins = ٩L),
    lbp_features(gray, n_bins = ١٦L),
    intensity_stats(gray)
  )
}
```

```
بارگذاری تصاویر #



load_split <- function(split_name, n_per_class) {
  records <- list()
  for (cls in CLASS_NAMES) {
    folder <- file.path(DATA_ROOT, split_name, cls)
    if (!dir.exists(folder)) stop(paste("پوشه یافت نشد:", folder))


    imgs <- list.files(folder, pattern = "\\.(jpg|jpeg|png)$",
                       full.names = TRUE, ignore.case = TRUE)
    if (length(imgs) == ۰) stop(paste("هیچ تصویری در", folder, "نیست"))


    sel <- if (is.infinite(n_per_class) || length(imgs) <= n_per_class) imgs else
sample(imgs, n_per_class)


    cat(sprintf("  [%s/%s] %d تصویر پردازش می‌شود...\n", split_name, cls, length(sel)))


    ok <- ۰L
    for (p in sel) {
      f <- extract_one(p)
      if (!is.null(f)) {
```

```
      records[[length(records) + ١]] <- c(label = which(CLASS_NAMES == cls),
                                          label_name = cls, f)
      ok <- ok + ١L
    }
  }
  cat(sprintf("  \u٢٧١٣ %d/%d موفق\n", ok, length(sel)))
}

df <- as.data.frame(do.call(rbind, records))
df$label      <- as.integer(df$label)
df$label_name <- as.character(df$label_name)
feat_cols <- paste٠("f", seq_len(ncol(df) - ٢))
names(df)[٣:ncol(df)] <- feat_cols
for (fc in feat_cols) df[[fc]] <- as.numeric(df[[fc]])
df
}

split_train_val <- function(df, val_frac = VAL_FRAC, seed = ١٤٠٢) {
set.seed(seed)
idx_val <- unlist(lapply(split(seq_len(nrow(df)), df$label_name), function(idx) {
  n_val <- max(١L, round(length(idx) * val_frac))
  sample(idx, n_val)
}))
list(train = df[-idx_val, ], val = df[idx_val, ])
```

```
}

df_train_full <- load_split("Training", n_per_class = N_PER_CLASS)
df_test      <- load_split("Testing",  n_per_class = N_TEST_CLASS)

sp <- split_train_val(df_train_full)
df_train <- sp$train
df_val   <- sp$val
rm(sp); gc(FALSE)

cat(sprintf("\nTrain: %d | Val: %d | Test: %d نمونه\n",
          nrow(df_train), nrow(df_val), nrow(df_test)))
cat("توزیع کلاس‌ها (Train):\n"); print(table(df_train$label_name))
cat("توزیع کلاس‌ها (Val):\n");   print(table(df_val$label_name))
cat("توزیع کلاس‌ها (Test):\n");  print(table(df_test$label_name))


#  PCA


feat_cols <- grep("^f[۰-۹]+$", names(df_train), value = TRUE)
pca_model <- prcomp(df_train[, feat_cols], center = TRUE, scale. = TRUE)

cum_var <- cumsum(pca_model$sdev^۲) / sum(pca_model$sdev^۲)
```

```
pca_plot_df <- data.frame(
  PC = seq_along(pca_model$sdev),
  Variance = (pca_model$sdev^٢) / sum(pca_model$sdev^٢),
  CumulativeVariance = cum_var
)

X_train_full_pca <- predict(pca_model, df_train[, feat_cols])
X_val_full_pca   <- predict(pca_model, df_val[,   feat_cols])
X_test_full_pca  <- predict(pca_model, df_test[,  feat_cols])

max_pc  <- ncol(X_train_full_pca)
pc_grid <- unique(pmin(c(١٠, ٢٠, ٣٠, ٥٠, ٨٠, ١٢٠, ١٦٠), max_pc))

val_acc_by_pc <- sapply(pc_grid, function(k) {
  Xtr_k <- scale(X_train_full_pca[, ١:k, drop = FALSE])
  ctr <- attr(Xtr_k, "scaled:center"); scl <- attr(Xtr_k, "scaled:scale")
  Xv_k <- scale(X_val_full_pca[, ١:k, drop = FALSE], center = ctr, scale = scl)

  fit <- tryCatch(
    nnet::multinom(y ~ ., data = data.frame(y = df_train$label_name, Xtr_k),
                   trace = FALSE, MaxNWts = ٢٠٠٠٠, maxit = ٣٠٠),
    error = function(e) NULL
  )
```

```
  if (is.null(fit)) return(NA_real_)
  pred <- predict(fit, newdata = data.frame(Xv_k))
  mean(pred == df_val$label_name) * ۱۰۰
})
names(val_acc_by_pc) <- pc_grid
print(round(val_acc_by_pc, ۲))


n_pc <- as.integer(pc_grid[which.max(val_acc_by_pc)])
cat(sprintf("PC انتخاب‌شده (بر اساس Validation): %d  (Val acc=%.۲f%%, واریانس
۱٪=تجمعیf%%)\n",
          n_pc, max(val_acc_by_pc, na.rm = TRUE), cum_var[n_pc] * ۱۰۰))


p_scree <- ggplot(pca_plot_df[۱:min(nrow(pca_plot_df), max(n_pc * ۳, ٤۰)), ],
                  aes(x = PC, y = Variance)) +
  geom_line(color = "#٤C۷۸A۸", linewidth = ۱,۱) +
  geom_point(color = "#٤C۷۸A۸", size = ۲,۲) +
  geom_vline(xintercept = n_pc, linetype = "dashed", color = "red", linewidth = ۰,۹)
+
  annotate("text", x = n_pc, y = pca_plot_df$Variance[n_pc],
           label = paste۰("Selected PC = ", n_pc), hjust = -۰,۱, vjust = -۰,۷,
           size = ٤, color = "red") +
  scale_y_continuous(labels = percent_format(accuracy = ۱)) +
  labs(title = "Scree Plot of PCA", x = "Principal Component", y = "Explained
Variance") +
  theme_minimal(base_size = ۱۳)
```

```
print(p_scree)


p_cumvar <- ggplot(pca_plot_df[١:min(nrow(pca_plot_df), max(n_pc * ٣, ٤٠)), ],
                   aes(x = PC, y = CumulativeVariance)) +
  geom_line(color = "#٢ca٠٢c", linewidth = ١,١) +
  geom_point(color = "#٢ca٠٢c", size = ٢,٥) +
  geom_vline(xintercept = n_pc, linetype = "dashed", color = "darkgreen",
linewidth = ١) +
  geom_hline(yintercept = cum_var[n_pc], linetype = "dashed", color = "gray٥٠") +
  annotate("point", x = n_pc, y = cum_var[n_pc], color = "darkgreen", size = ٣,٥) +
  annotate("text", x = n_pc + ١, y = cum_var[n_pc] - ٠,٠٤,
           label = paste٠("Selected PC = ", n_pc, "\nCum. Var = ",
round(cum_var[n_pc]*١٠٠, ١), "%"),
           color = "darkgreen", hjust = ٠, fontface = "bold") +
  scale_y_continuous(labels = percent_format(accuracy = ١)) +
  labs(title = "Cumulative Explained Variance by PCA (zoom)", x = "Principal
Component",
       y = "Cumulative Explained Variance") +
  theme_minimal(base_size = ١٣) + theme(panel.grid.minor = element_blank())
print(p_cumvar)


X_tr_pca  <- X_train_full_pca[, ١:n_pc, drop = FALSE]
X_val_pca <- X_val_full_pca[,  ١:n_pc, drop = FALSE]
X_te_pca  <- X_test_full_pca[, ١:n_pc, drop = FALSE]
rm(X_train_full_pca, X_val_full_pca, X_test_full_pca); gc(FALSE)
```

```
X_sc  <- scale(X_tr_pca)
sc_c٢ <- attr(X_sc, "scaled:center")
sc_s٢ <- attr(X_sc, "scaled:scale")

X_train <- cbind(١, X_sc)
X_val   <- cbind(١, scale(X_val_pca, center = sc_c٢, scale = sc_s٢))
X_test  <- cbind(١, scale(X_te_pca,  center = sc_c٢, scale = sc_s٢))
colnames(X_train)[١] <- colnames(X_val)[١] <- colnames(X_test)[١] <- "intercept"

make_Y <- function(df) {
  Y <- matrix(٠L, nrow(df), C, dimnames = list(NULL, CLASS_NAMES))
  for (j in seq_len(C)) Y[, j] <- as.integer(df$label_name == CLASS_NAMES[j])
  Y
}
Y_train <- make_Y(df_train)
Y_val   <- make_Y(df_val)
Y_test  <- make_Y(df_test)

N <- nrow(X_train); P <- ncol(X_train); ref_cat <- C

cat(sprintf("N=%d | P=%d (intercept+%d PC) | C=%d\n", N, P, n_pc, C))
cat(sprintf("پارامتر کل: %d × %d = %d\n", P, C - ١, P * (C - ١)))
```

```
rm(X_tr_pca, X_val_pca, X_te_pca); gc(FALSE)



n_sample <- ٦٠٠٠L
n_burn   <- ٦٠٠٠L
THIN     <- ٥L
prior_mu  <- rep(٠, P)
prior_cov <- diag(١, P)

cat(sprintf("\nMCMC : n_sample=%d | n_burn=%d | thin=%d\n",
          n_sample, n_burn, THIN))

results <- list()

run_timed <- function(name, expr) {
  cat(sprintf("\n\u٢٥٠١\u٢٥٠١\u٢٥٠١ %s \u٢٥٠١\u٢٥٠١\u٢٥٠١\n", name))
  set.seed(١٤٠٢)
  tic <- proc.time()
  out <- eval(expr)
  rt  <- (proc.time() - tic)["elapsed"]
  cat(sprintf("  \u٢٧١٣ %.١f ثانيه (%.١f دقيقه)\n", rt, rt / ٦٠))
  gc(FALSE)
  list(out = out, runtime = rt, name = name)
}
```

```
# Pólya-Gamma


fit_binary_PG <- function(y, X, n_sample, n_burn, prior_mean, prior_var) {
  n <- nrow(X); P <- ncol(X)
  B_inv  <- solve(prior_var)
  Binv_b <- as.numeric(B_inv %*% prior_mean)
  kappa  <- y - ۰٫۵
  beta   <- rep(۰, P)
  n_iter <- n_sample + n_burn
  draws  <- matrix(NA_real_, nrow = P, ncol = n_sample)

  for (iter in seq_len(n_iter)) {
    psi   <- as.numeric(X %*% beta)
    omega <- BayesLogit::rpg(n, h = ۱, z = psi)

    Xw    <- X * omega
    V_inv <- crossprod(X, Xw) + B_inv
    V     <- chol۲inv(chol(V_inv))
    m     <- as.numeric(V %*% (crossprod(X, kappa) + Binv_b))
    beta  <- as.numeric(m + t(chol(V)) %*% rnorm(P))
```

```
    if (iter > n_burn) draws[, iter - n_burn] <- beta
  }
  draws
}

multilogit_PG_stickbreak <- function(Y, X, n_sample, n_burn, prior_mean,
prior_var,
                                     reference_cat, progress = TRUE) {
  C <- ncol(Y); P <- ncol(X)
  cls_int   <- max.col(Y)
  order_vec <- c(setdiff(seq_len(C), reference_cat), reference_cat)
  K <- C - ١L
  posterior_coef <- array(٠, dim = c(P, C, n_sample))

  for (k in seq_len(K)) {
    target_cls  <- order_vec[k]
    active_mask <- cls_int %in% order_vec[k:C]
    y_k <- as.numeric(cls_int[active_mask] == target_cls)
    X_k <- X[active_mask, , drop = FALSE]

    if (progress) cat(sprintf("  [PG] مرحله %d/%d — هدف کلاس='%s' | فعال نمونهٔ=%d\n",
                              k, K, CLASS_NAMES[target_cls], nrow(X_k)))

    draws_k <- fit_binary_PG(y_k, X_k, n_sample, n_burn, prior_mean, prior_var)
    posterior_coef[, target_cls, ] <- draws_k
```

```
    rm(draws_k, X_k, y_k); gc(FALSE)
  }
  list(posterior_coef = posterior_coef, method = "stickbreak", order = order_vec)
}

predict_cls_stickbreak <- function(pc, X, order_vec, ref_cat, C) {
  bm <- apply(pc, c(١, ٢), mean)
  n  <- nrow(X)
  prob      <- matrix(٠, n, C)
  remaining <- rep(١, n)
  K <- C - ١L
  for (k in seq_len(K)) {
    cls   <- order_vec[k]
    eta_k <- as.numeric(X %*% bm[, cls])
    p_k   <- ١ / (١ + exp(-eta_k))
    prob[, cls] <- remaining * p_k
    remaining   <- remaining * (١ - p_k)
  }
  prob[, order_vec[C]] <- remaining
  max.col(prob)
}


results[["AMH"]] <- run_timed("AMH — Adaptive Metropolis-Hastings", quote(
```

```
  multilogit(Y_train, X_train, n_sample = n_sample, n_burn = n_burn,
             n_sigma_check = ٢٥L, step_size = ٠,٠٥,
             prior = "normal", prior_mean = prior_mu, prior_var = prior_cov,
             reference_cat = ref_cat, probs = FALSE, progress = TRUE)
))

results[["DA+AMH"]] <- run_timed("DA+AMH — Gamma Augmentation +
Adaptive MH", quote(
  multilogit(Y_train, X_train, n_sample = n_sample, n_burn = n_burn,
             n_sigma_check = ٢٥L, step_size = ٠,١,
             prior = "normal", prior_mean = prior_mu, prior_var = prior_cov,
             reference_cat = ref_cat, probs = FALSE, progress = TRUE)
))

results[["DA+eSS"]] <- run_timed("DA+eSS — Gamma Augmentation + Elliptical
Slice Sampling", quote(
  multilogit_ESS(Y_train, X_train, n_sample = n_sample, n_burn = n_burn,
                 prior_mean = prior_mu, prior_var = prior_cov,
                 reference_cat = ref_cat, probs = FALSE, progress = TRUE)
))

results[["PG"]] <- run_timed("PG — Pólya-Gamma Bayesian Logit (Polson et al.
٢٠١٣)", quote(
  multilogit_PG_stickbreak(Y_train, X_train, n_sample = n_sample, n_burn =
n_burn,
                           prior_mean = prior_mu, prior_var = prior_cov,
```

```
                    reference_cat = ref_cat, progress = TRUE)
))


for (nm in names(results)) {
  obj <- results[[nm]]$out
  if (is.list(obj) && !is.null(obj$posterior_coef)) {
    n_draws <- dim(obj$posterior_coef)[٣]
    keep <- seq(١, n_draws, by = THIN)
    obj$posterior_coef <- obj$posterior_coef[, , keep, drop = FALSE]
    results[[nm]]$out <- obj
    cat(sprintf("  [thin] %s: %d -> %d نمونه\n", nm, n_draws, length(keep)))
  }
}



is_mcmc_method <- function(res) {
  is.list(res$out) && !is.null(res$out$posterior_coef)
}

get_mcmc_perf <- function(res) {
  rt <- res$runtime
  pc <- res$out$posterior_coef
```

```
  active <- setdiff(seq_len(C), ref_cat)
  ess_v <- unlist(lapply(active, function(j)
    lapply(seq_len(dim(pc)[١]), function(p)
      tryCatch(as.numeric(coda::effectiveSize(coda::mcmc(pc[p, j, ]))),
               error = function(e) NA_real_))))
  ess_v <- ess_v[!is.na(ess_v) & is.finite(ess_v)]
  if (length(ess_v) == ٠) {
    return(data.frame(Method = res$name, Runtime_s = round(rt, ١),
                      Min_ESS = NA_real_, Med_ESS = NA_real_,
                      Min_ESR = NA_real_, Med_ESR = NA_real_))
  }
  data.frame(Method = res$name, Runtime_s = round(rt, ١),
             Min_ESS = round(min(ess_v), ١), Med_ESS = round(median(ess_v), ١),
             Min_ESR = round(min(ess_v) / rt, ٣), Med_ESR = round(median(ess_v) / rt,
٣))
}

mcmc_results <- results[sapply(results, is_mcmc_method)]
mcmc_perf <- do.call(rbind, lapply(mcmc_results, get_mcmc_perf))
rownames(mcmc_perf) <- NULL
print(mcmc_perf)



posterior_methods <- names(results)[sapply(results, is_mcmc_method)]
```

```
mcols <- c("AMH" = "#٢C٣E٥٠", "DA+AMH" = "#E٦٧E٢٢", "DA+eSS" = "#٢٩٨٠B٩",
"PG" = "#٢٧AE٦٠")

method_labels <- setNames(posterior_methods, posterior_methods)


target_class <- ١L  # کلاس هدف برای دیاگنوستیک = glioma

P_check <- min(٦L, dim(results[[posterior_methods[١]]]$out$posterior_coef)[١])

coef_idx <- seq_len(P_check)

coef_names <- c("intercept", paste٠("PC", seq_len(P_check - ١)))


cat(sprintf("بررسی همگرایی برای کلاس هدف='%s' | ضرایب: %s\n",

        CLASS_NAMES[target_class], paste(coef_names, collapse = ", ")))


for (m in posterior_methods) {

 pc <- results[[m]]$out$posterior_coef

 n_row <- ceiling(P_check / ٢)

 par(mfrow = c(n_row, ٢), mar = c(٤, ٤, ٣, ١), oma = c(٠, ٠, ٢, ٠))

 for (k in seq_along(coef_idx)) {

  ch <- pc[coef_idx[k], target_class, ]

  plot(ch, type = "l", col = unname(mcols[m]), lwd = ٠,٦, main = coef_names[k],

      xlab = "Iteration", ylab = expression(beta))

  abline(h = mean(ch), col = "red", lty = ٢, lwd = ١,٣)

  if (length(ch) > ٥٠) lines(cumsum(ch) / seq_along(ch), col = "orange", lwd = ١,٢)

 }

 mtext(sprintf("Trace — %s (class: %s)", results[[m]]$name,
CLASS_NAMES[target_class]),
```

```
      outer = TRUE, cex = ١,٠, font = ٢)
  par(mfrow = c(١, ١))
}

for (m in posterior_methods) {
  pc <- results[[m]]$out$posterior_coef
  n_row <- ceiling(P_check / ٢)
  par(mfrow = c(n_row, ٢), mar = c(٤, ٤, ٣, ١), oma = c(٠, ٠, ٢, ٠))
  for (k in seq_along(coef_idx)) {
    acf(pc[coef_idx[k], target_class, ], main = coef_names[k], col =
unname(mcols[m]), lag.max = ٥٠)
  }
  mtext(sprintf("ACF — %s (class: %s)", results[[m]]$name,
CLASS_NAMES[target_class]),
      outer = TRUE, cex = ١,٠, font = ٢)
  par(mfrow = c(١, ١))
}

diag_table <- do.call(rbind, lapply(posterior_methods, function(m) {
  pc <- results[[m]]$out$posterior_coef
  n_iter <- dim(pc)[٣]
  do.call(rbind, lapply(seq_along(coef_idx), function(k) {
    ch <- pc[coef_idx[k], target_class, ]
    mc <- coda::mcmc(ch)
```

```
  acf۱ <- tryCatch(acf(ch, plot = FALSE, lag.max = ۱)$acf[۲, ۱, ۱], error =
function(e) NA_real_)
  ess  <- tryCatch(as.numeric(coda::effectiveSize(mc)), error = function(e)
NA_real_)
  gz   <- tryCatch(coda::geweke.diag(mc)$z, error = function(e) NA_real_)
  data.frame(Method = m, Coef = coef_names[k], ACF_lag۱ = round(acf۱, ۳),
             ESS = round(ess, ۱), ESS_ratio = round(ess / n_iter, ۳),
             Geweke_z = round(gz, ۲),
             Converged = ifelse(!is.na(gz) && abs(gz) < ۲, "بررسی شود" ,"بله"))
 }))
}))
print(diag_table, row.names = FALSE)


summary_by_method <- aggregate(cbind(ACF_lag۱, ESS_ratio, Geweke_z) ~
Method, data = diag_table,
                               FUN = function(x) mean(abs(x), na.rm = TRUE))
names(summary_by_method) <- c("Method", "Mean_|ACF_lag۱|",
"Mean_ESS_ratio", "Mean_|Geweke_z|")
print(summary_by_method, row.names = FALSE)



softmax <- function(eta) {
 eta <- sweep(eta, ۱, apply(eta, ۱, max), "-")
 exp_eta <- exp(eta)
 exp_eta / rowSums(exp_eta)
```

```
}

predict_cls_posterior_coef <- function(pc, X) {
  bm <- apply(pc, c(١, ٢), mean)
  eta <- X %*% bm
  eta[, ref_cat] <- ٠
  max.col(softmax(eta))
}

predict_cls_any <- function(res, X) {
  obj <- res$out
  if (is.list(obj) && !is.null(obj$posterior_coef)) {
    if (!is.null(obj$method) && obj$method == "stickbreak") {
      return(predict_cls_stickbreak(obj$posterior_coef, X, obj$order, ref_cat, C))
    }
    return(predict_cls_posterior_coef(obj$posterior_coef, X))
  }
  stop(paste("Prediction not implemented for:", res$name))
}

true_train <- apply(Y_train, ١, which.max)
true_val   <- apply(Y_val,   ١, which.max)
true_test  <- apply(Y_test,  ١, which.max)
```

```
compute_acc <- function(res, X, true_cls) {
  tryCatch({
    pred <- predict_cls_any(res, X)
    mean(pred == true_cls) * ١٠٠
  }, error = function(e) {
    message("Prediction failed for: ", res$name, " | ", conditionMessage(e))
    NA_real_
  })
}

overfit_df <- do.call(rbind, lapply(names(results), function(nm) {
  data.frame(
    Method = results[[nm]]$name, Key = nm,
    Train = round(compute_acc(results[[nm]], X_train, true_train), ٢),
    Val   = round(compute_acc(results[[nm]], X_val,   true_val),   ٢),
    Test  = round(compute_acc(results[[nm]], X_test,  true_test),  ٢)
  )
}))
overfit_df$Gap_Train_Test <- round(overfit_df$Train - overfit_df$Test, ٢)
overfit_df$Gap_Train_Val  <- round(overfit_df$Train - overfit_df$Val, ٢)
rownames(overfit_df) <- NULL

cat("\nجدول دقت Train/Val/Test و شکاف Overfitting:\n")
print(overfit_df)
```

```
cat("\nهرچه به صفر نزدیک‌تر باشند، Gap_Train_Val و Gap_Train_Test راهنما:\n")

cat("مدل روی داده‌های دیده‌نشده هم خوب تعمیم می‌یابد (overfit کمتر).\n")


best_m <- overfit_df$Key[which.max(overfit_df$Test)]

pred_test_best <- predict_cls_any(results[[best_m]], X_test)

pred_val_best  <- predict_cls_any(results[[best_m]], X_val)


cm_test <- table(Predicted = CLASS_NAMES[pred_test_best], Actual =
CLASS_NAMES[true_test])

cm_val  <- table(Predicted = CLASS_NAMES[pred_val_best],  Actual =
CLASS_NAMES[true_val])

cat(sprintf("\nConfusion Matrix (Test) — %s:\n", results[[best_m]]$name));
print(cm_test)

cat(sprintf("\nConfusion Matrix (Validation) — %s:\n", results[[best_m]]$name));
print(cm_val)


overfit_long <- overfit_df %>%

  dplyr::select(Key, Train, Val, Test) %>%

  tidyr::pivot_longer(cols = c(Train, Val, Test), names_to = "Split", values_to =
"Accuracy") %>%

  dplyr::mutate(Split = factor(Split, levels = c("Train", "Val", "Test")))


p_overfit <- ggplot(overfit_long, aes(x = Key, y = Accuracy, fill = Split)) +

  geom_col(position = position_dodge(۰٫۷۵), width = ۰٫۶۵) +

  geom_text(aes(label = paste۰(round(Accuracy, ۱), "%")),
```

```
        position = position_dodge(۰٫۷۵), vjust = -۰٫٤, size = ۳٫۳) +
  scale_fill_manual(values = c("Train" = "#۹۵A۵A٦", "Val" = "#F۳۹C۱۲", "Test" =
"#۲۹۸۰B۹")) +
  coord_cartesian(ylim = c(۰, ۱۰۵)) +
  labs(title = "دقت روی Train / Validation / Test — بررسی Overfitting",
       subtitle = "نشانهٔ overfitting = Val/Test از Train فاصلهٔ زیاد",
       x = NULL, y = "Accuracy (%)") +
  theme_minimal(base_size = ۱۲) +
  theme(plot.title = element_text(hjust = ۰٫۵, face = "bold"),
        plot.subtitle = element_text(hjust = ۰٫۵, color = "gray٤۰"),
        legend.position = "bottom")
print(p_overfit)

plot_perf <- mcmc_perf
plot_perf$Key <- names(mcmc_results)

existing_keys <- unique(plot_perf$Key)
mcols_use <- mcols[names(mcols) %in% existing_keys]

th <- theme_minimal(base_size = ۱۲) +
  theme(plot.title = element_text(hjust = ۰٫۵, face = "bold"),
        plot.subtitle = element_text(hjust = ۰٫۵, color = "gray٤۰"),
        legend.position = "bottom", axis.text.x = element_text(angle = ۲۵, hjust = ۱))
```

```
p١ <- plot_perf %>%
  dplyr::select(Key, Min_ESS, Med_ESS) %>%
  tidyr::pivot_longer(c(Min_ESS, Med_ESS), names_to = "Type", values_to = "ESS")
%>%
  dplyr::mutate(Type = dplyr::recode(Type, "Min_ESS" = "Minimum", "Med_ESS" =
"Median")) %>%
  ggplot(aes(x = Key, y = ESS, fill = Key, alpha = Type)) +
  geom_col(position = "dodge", width = ٠,٧) +
  scale_x_discrete(labels = method_labels) + scale_fill_manual(values =
mcols_use) +
  scale_alpha_manual(values = c("Minimum" = ٠,٥, "Median" = ١,٠)) +
  facet_wrap(~Type, scales = "free_y") +
  labs(title = "Effective Sample Size (ESS)",
       subtitle = sprintf("Brain MRI | C=%d | N=%d | P=%d", C, N, P), x = NULL, y =
"ESS") +
  th + guides(fill = "none", alpha = "none")
print(p١)


p٢ <- plot_perf %>%
  dplyr::select(Key, Min_ESR, Med_ESR) %>%
  tidyr::pivot_longer(c(Min_ESR, Med_ESR), names_to = "Type", values_to = "ESR")
%>%
  dplyr::mutate(Type = dplyr::recode(Type, "Min_ESR" = "Minimum", "Med_ESR" =
"Median"),
                lESR = log١٠(pmax(ESR, ١e-٥))) %>%
  ggplot(aes(x = Key, y = lESR, fill = Key, alpha = Type)) +
```

```
  geom_col(position = "dodge", width = ٠,٧) +
  geom_hline(yintercept = ٠, linetype = "dashed", color = "gray٥٠") +
  scale_x_discrete(labels = method_labels) + scale_fill_manual(values =
mcols_use) +
  scale_alpha_manual(values = c("Minimum" = ٠,٥, "Median" = ١,٠)) +
  facet_wrap(~Type) +
  labs(title = "log١٠(ESR) — Higher = Better Efficiency",
       subtitle = "ESR = Effective Sample Size per second", x = NULL, y =
"log١٠(ESS/s)") +
  th + guides(fill = "none", alpha = "none")
print(p٢)

p٣ <- ggplot(plot_perf, aes(x = Key, y = Runtime_s, fill = Key)) +
  geom_col(width = ٠,٦, alpha = ٠,٩) +
  geom_text(aes(label = paste٠(Runtime_s, "s")), vjust = -٠,٣, size = ٤) +
  scale_x_discrete(labels = method_labels) + scale_fill_manual(values =
mcols_use) +
  labs(title = "Runtime", subtitle = "Lower is faster", x = NULL, y = "Seconds") +
  th + guides(fill = "none")
print(p٣)

p٤ <- ggplot(overfit_df, aes(x = Key, y = Test, fill = Key)) +
  geom_col(width = ٠,٦, alpha = ٠,٩, na.rm = TRUE) +
  geom_text(aes(label = paste٠(Test, "%")), vjust = -٠,٣, size = ٤, na.rm = TRUE) +
  scale_x_discrete(labels = method_labels) + scale_fill_manual(values =
mcols_use) +
```

```
  coord_cartesian(ylim = c(٠, ١٠٠)) +
  labs(title = "Test Set Accuracy", subtitle = "Higher is better", x = NULL, y =
"Accuracy (%)") +
  th + guides(fill = "none")
print(p٤)

grid.arrange(p١, p٢, p٣, p٤, nrow = ٢,
             top = "Comparison of Bayesian Multinomial Logit Methods — Brain Tumor
MRI")

n_feat_plot <- min(٧, P)
ci_list <- lapply(posterior_methods, function(m) {
  cm٢ <- results[[m]]$out$posterior_coef[١:n_feat_plot, ١, , drop = FALSE]
  cm٢_mat <- matrix(cm٢, nrow = n_feat_plot, ncol =
dim(results[[m]]$out$posterior_coef)[٣])
  feature_names <- c("intercept", paste٠("PC", ١:(n_feat_plot - ١)))
  data.frame(Key = m, Feature = feature_names, Mean = rowMeans(cm٢_mat),
             Lo = apply(cm٢_mat, ١, quantile, probs = ٠,٠٢٥),
             Hi = apply(cm٢_mat, ١, quantile, probs = ٠,٩٧٥))
})
ci_df <- do.call(rbind, ci_list)

p٦ <- ggplot(ci_df, aes(x = Feature, y = Mean, color = Key, ymin = Lo, ymax = Hi)) +
  geom_hline(yintercept = ٠, linetype = "dashed", color = "gray٦٠") +
  geom_errorbar(position = position_dodge(٠,٥), width = ٠,٣, linewidth = ٠,٨) +
```

```
  geom_point(position = position_dodge(٠,٥), size = ٢,٥) +
  scale_color_manual(values = mcols_use, labels = method_labels) +
  coord_flip() +
  labs(title = "Posterior ٩٥٪ Credible Intervals",
       subtitle = "Class: glioma | Baseline: pituitary", x = NULL, y = expression(beta))
+
  th
print(p٦)


final_table <- merge(mcmc_perf %>% dplyr::mutate(Key = names(mcmc_results)),
                     overfit_df[, c("Key", "Train", "Val", "Test", "Gap_Train_Test")],
                     by = "Key", all.x = TRUE, sort = FALSE)
final_table <- final_table[match(names(mcmc_results), final_table$Key), ]
final_table$Display <- final_table$Key


fmt_num <- function(x, digits = ١) ifelse(is.na(x), "NA", formatC(x, format = "f",
digits = digits))


for (i in seq_len(nrow(final_table))) {
  rw <- final_table[i, ]
  cat(sprintf("\u٢٥٥١ %-١٠s \u٢٥٥١ %٨s \u٢٥٥١ %٧s \u٢٥٥١ %٧s \u٢٥٥١ %٧s
\u٢٥٥١ %٧s \u٢٥٥١ %٧s \u٢٥٥١ %٧s \u٢٥٥١ %٧s \u٢٥٥١\n",
              rw$Display, fmt_num(rw$Runtime_s, ١), fmt_num(rw$Min_ESS, ١),
fmt_num(rw$Med_ESS, ١),
              fmt_num(rw$Min_ESR, ٣), fmt_num(rw$Med_ESR, ٣), fmt_num(rw$Train,
٢), fmt_num(rw$Val, ٢),
```

```
        fmt_num(rw$Test, ۲), fmt_num(rw$Gap_Train_Test, ۲)))
}

idx_fast    <- which.min(final_table$Runtime_s)
idx_med_ess <- which.max(final_table$Med_ESS)
idx_min_ess <- which.max(final_table$Min_ESS)
idx_med_esr <- which.max(final_table$Med_ESR)
idx_test    <- which.max(final_table$Test)
idx_gap     <- which.min(abs(final_table$Gap_Train_Test))

cat(sprintf("  • سریع‌ترین روش: %s (%.۱f ثانیه)\n", final_table$Display[idx_fast],
final_table$Runtime_s[idx_fast]))
cat(sprintf("  • بهترین Median ESS: %s (%.۱f)\n", final_table$Display[idx_med_ess],
final_table$Med_ESS[idx_med_ess]))
cat(sprintf("  • پایدارترین Min ESS: %s (%.۱f)\n", final_table$Display[idx_min_ess],
final_table$Min_ESS[idx_min_ess]))
cat(sprintf("  • کاراترین Median ESR: %s (%.۳f)\n", final_table$Display[idx_med_esr],
final_table$Med_ESR[idx_med_esr]))
cat(sprintf("  • بهترین دقت روی Test: %s (%.۲f%%)\n", final_table$Display[idx_test],
final_table$Test[idx_test]))
cat(sprintf("  • کمترین Overfitting (Gap Train-Test): %s (%.۲f%%)\n",
final_table$Display[idx_gap], final_table$Gap_Train_Test[idx_gap]))
```